



University of Jeddah
College of Computer Science and Engineering

CCSW 325 Software Construction

Analyzing Identifier Naming and Code Understanding Using AI

Prepared by:

Abrar Alharbi	2317129
Ghala Alharbi	2310582
Elaf Alsharif	2310115
Tooleen Wael	2311776
Khadija janbi	2312308

Submitted to: Wafa

March/2026



جامعة جدة
University of Jeddah

University of Jeddah
College of Computer Science and Engineering

Phase ONE

1. Introduction

In software development, writing readable and understandable code is essential for maintaining and improving software systems. One important factor that influences code readability is the quality of identifier names, such as variable names, function names, and class names. These identifiers help developers understand the purpose and behavior of the code. When identifiers are meaningful and descriptive, programmers can comprehend the code more quickly and accurately. However, when identifiers are unclear, too short, or meaningless, understanding the code becomes more difficult.

Several studies in software engineering have shown that identifier names play a significant role in program comprehension. Developers often rely on identifier names to understand the intent of the code, especially when comments or documentation are limited. Poor naming practices may lead to confusion, increased development time, and difficulties in software maintenance. As a result, improving identifier naming has become an important research topic in software engineering.

In recent years, researchers have explored the use of Artificial Intelligence (AI) and Large Language Models (LLMs) to assist in code analysis and generation tasks. These models are capable of analyzing large amounts of source code and learning common programming patterns. Consequently, they can be used to suggest meaningful identifier names, summarize code, and support developers in understanding complex programs.

This project focuses on studying identifier naming in source code and reviewing existing research related to variable naming, code readability, and AI-based code analysis. The goal is to understand how identifier naming affects code comprehension and how modern AI techniques can help generate better variable names. By analyzing several research papers, this study aims to highlight the importance of identifier naming and explore the potential role of AI in improving software readability and development practices.

1- Literature Review

Recent studies have explored the importance of identifier naming and its impact on program comprehension. Researchers have shown that meaningful variable names can significantly improve code readability and help developers understand software systems more efficiently. In addition, several studies have investigated the use of machine learning and large language models (LLMs) to generate or recover meaningful identifier names in source code. These approaches aim to support developers by improving code clarity and assisting in software maintenance and analysis.

To better understand the significance of identifier naming in software development, several studies have reviewed existing research on naming practices and their impact on program comprehension.

2.1- Background on Identifier Naming Research

This paper presents a literature review on identifier names used in computer programs [1]. The study analyzes previous research to understand how identifier names affect program comprehension and software development. The review includes 33 research papers published between 1998 and 2021 that focus on different aspects of identifier naming in software engineering.

The authors classify the existing research into three main areas. The first area studies the structure and semantics of identifiers, focusing on how developers construct identifier names and how these names represent program behavior. The second area focuses on measuring identifier quality by proposing different models and guidelines that help determine whether identifier names are clear, consistent, and meaningful. The third research area investigates how developers choose identifier names and the cognitive and practical factors that influence naming decisions during software development.

The findings of the review indicate that identifier names play an important role in program comprehension and software quality. Many studies show that well-structured and meaningful identifiers can improve developers' understanding of code and increase productivity. However, the paper also highlights that tools and naming guidelines that support better identifier naming are rarely used in real software development environments. The study concludes that more research is needed to develop comprehensive theories and tools that can improve identifier naming practices and support program comprehension.

Reflection:

This paper provides a comprehensive overview of identifier naming research and highlights the importance of meaningful identifiers in software engineering. One of its main contributions is organizing previous studies into different research categories, which helps researchers better understand the field. However, the study mainly focuses on reviewing existing literature and does not propose practical AI-

based solutions for improving identifier naming. Future research could focus on developing intelligent tools that automatically evaluate and improve identifier quality in real software projects.

Building on this general understanding of identifier naming research, several empirical studies have specifically examined how naming practices influence programmers' ability to understand source code.

2.2- The Role of Identifier Naming in Program Comprehension

This study examines the impact of identifier naming on program comprehension [2]. In software development, programmers typically rely on two main sources of information to understand code: identifier names and comments. However, many programs lack sufficient comments, making identifier names a primary source for understanding the functionality of the code. Therefore, this study aims to analyze how the quality of identifier names affects developers' ability to comprehend source code.

The researchers conducted an experiment involving more than 100 programmers, including both students and professionals in computer science. Participants were shown a set of functions with three different styles of variable names: single-letter identifiers, abbreviated names, and full English words. They were asked to read the code, describe the function's purpose, and rate their confidence in their understanding. The collected data was analyzed using statistical models to evaluate the effect of identifier naming as well as other factors such as experience and education.

The results showed that identifiers written using full English words led to the highest level of code comprehension compared to single-letter names. Although abbreviations were sometimes understandable, they were generally less clear than full-word identifiers. The study concludes that using meaningful and descriptive identifier names can significantly improve code readability and support developers in understanding and maintaining software systems.

Reflection:

This study demonstrates that meaningful identifier names significantly improve program comprehension and help developers understand source code more efficiently. The experimental design involving students and professional developers strengthens the reliability of the findings. However, the experiment used relatively small code examples, which may not fully represent large real-world software systems. Future work could investigate how identifier naming affects comprehension in large-scale industrial projects and collaborative development environments.

Similarly, another study further explored the role of code structure and intermediate variables in improving program comprehension.

This paper explores how the structure of source code and the naming of variables affect the way programmers understand code [3]. Code comprehension is a very important part of software

development because developers often need to read and analyze existing code before modifying it. If the code is difficult to understand, it can slow down development and increase the risk of errors. The main focus of the study is on the use of intermediate variables. These are temporary variables that break complex expressions into smaller steps. The researchers wanted to find out whether using intermediate variables helps programmers understand code more easily, and whether meaningful variable names make a difference in comprehension.

To investigate this question, the researchers designed an experiment where participants were asked to read several small functions and identify their purpose. Each function was presented in three different versions. The first version used a single complex expression without intermediate variables. The second version broke the expression into smaller steps using intermediate variables with meaningless names such as temporary labels. The third version used the same structure but assigned meaningful names to the variables.

Participants in the experiment were asked to read the code and suggest a name for the function that describes what it does. If the participant could correctly name the function, it meant they understood the functionality of the code. The researchers also measured the time participants needed to reach the correct answer in order to evaluate how difficult the code was to understand.

The code examples used in the experiment consisted of several mathematical functions written in Python. These functions included operations such as checking whether a number is prime, sorting arrays, computing variance, and identifying right triangles. The researchers selected mathematical functions so that participants could focus on the structure of the code rather than needing specialized domain knowledge.

The results of the experiment showed that meaningful variable names improve code comprehension in many situations. Participants were more likely to correctly identify the purpose of a function when the intermediate variables had descriptive names. The results also showed that the benefit of intermediate variables becomes more noticeable when the code is more complex. In simpler functions, the difference between the versions was smaller.

Overall, the study emphasizes the importance of writing readable and well structured code.

Meaningful variable names can help programmers understand code more quickly and accurately.

Although the experiment was conducted using relatively simple functions, the findings suggest that good naming practices and clear structure can significantly improve code readability and maintainability

Reflection:

The study contributes to software readability research by showing that meaningful intermediate variables improve code understanding, especially in complex functions. The findings highlight the importance of code structure and naming practices in software maintenance. A limitation of the study is that it focused mainly on mathematical Python functions, which may limit the generalization of the results to other programming domains. Future research could evaluate the impact of intermediate variables in larger applications and different programming languages.

In addition to examining code structure, researchers have also investigated how the combination of meaningful variable names and intermediate variables can further enhance software comprehension.

This paper examines how code structure and variable naming affect the ability of programmers to understand software [4]. Code comprehension is a critical activity in software development

because developers frequently need to read and analyze existing code before modifying or extending it. If the code is difficult to understand, it can slow down development and increase the risk of mistakes. The researchers focus on how intermediate variables and their names influence the readability of code and the ability of programmers to identify what a program does.

To conduct the study, the researchers designed an experiment in which participants were asked to read several functions written in Python and determine the purpose of each function. Each function was presented in three different versions. In the first version, the function used a single complex expression without intermediate variables. In the second version, the same function was rewritten using intermediate variables with meaningless names. In the third version, the function used intermediate variables with descriptive and meaningful names. Participants were asked to identify the purpose of the function and the time required to reach the correct answer was recorded.

The results showed that meaningful variable names significantly improve code comprehension. Participants were more likely to correctly identify the purpose of a function when descriptive names were used for intermediate variables. The researchers also found that the benefit of

meaningful names becomes more noticeable as the complexity of the code increases. These findings highlight the importance of good naming practices in software development and demonstrate how code readability can directly influence a programmer's ability to understand and maintain software systems.

While the previous studies focused on how identifier naming affects human understanding of code, recent research has also examined how naming practices influence the performance of large language models in automated code analysis tasks.

2.3- The Impact of Identifier Naming on AI and Code Analysis

These studies investigate how identifier naming affects the performance of large language models in code analysis tasks [5]. Both papers examine whether meaningful identifier names help language models better understand source code and whether models rely heavily on naming information during code analysis.

In both studies, the researchers modified several code datasets by replacing meaningful identifier names with meaningless ones. Two main approaches were used: replacing identifiers with random strings and shuffling identifier names so that they no longer match the meaning of the code. The modified datasets were then used to evaluate the performance of the CodeBERT model on tasks such as code search and code clone detection.

One of the studies extended this experimental design by creating multiple versions of the datasets through systematic changes to different elements of the code, including variable names and method names. In total, sixteen dataset variations were generated to analyze how different types of identifier modifications affect model performance. The evaluation used metrics such as Mean Reciprocal Rank (MRR) for code search and F1 score for clone detection.

The results from both studies show that modifying or removing meaningful identifier names significantly reduces model performance, particularly in code search tasks. The findings also indicate that Python programs are more sensitive to naming changes than Java programs, likely because Python relies more heavily on descriptive variable names to support readability. In contrast, clone detection tasks were less affected by naming changes, possibly because they depend more on structural similarity than on identifier names.

Overall, the studies suggest that large language models rely strongly on identifier names when analyzing code. These findings highlight an important limitation in current models, which may depend more on lexical information than on a deeper understanding of program logic.

Reflection:

These studies provide important insights into how large language models depend heavily on identifier names during code analysis tasks. Their main contribution is demonstrating that modifying identifier names significantly affects model performance, particularly in code search tasks. However, the experiments mainly focused on CodeBERT and a limited number of datasets. Future research could

explore more advanced language models and investigate techniques that improve semantic understanding beyond lexical naming information.

The authors recommend that future research focus on developing models that better capture program semantics rather than relying mainly on naming patterns.

Following these findings, researchers have begun developing practical AI-based techniques designed to improve identifier naming and enhance code readability automatically.

2.4- AI-Based Techniques for Identifier Naming and Code Readability

This paper studies how artificial intelligence can improve code readability by automatically suggesting better names for variables [6]. In many software projects, developers sometimes use unclear or generic variable names, which makes the code harder to understand and maintain. Poor naming forces developers to spend more time reading and analyzing the code to understand its purpose.

The researchers focus on the problem of variable name repair. In this task, a variable name inside a C++ function is replaced with a placeholder token, and the model must generate a suitable and meaningful name based on the context of the code. The idea is that the model should understand how the variable is used in the function and suggest a name that reflects its role.

To create training data, the researchers built a dataset using the C++ part of The Stack, a large dataset of open-source code. They used a parsing tool to extract functions and replace one identifier in each function with a placeholder. The original name was used as the correct answer so the model could learn naming patterns from real code examples.

The system is built on the Llama 3.1–8B language model, which is fine tuned using a technique called LoRA. The researchers also added training strategies such as warmup and dropout to improve the stability of training.

In addition, they designed a reranking model that selects the best name from several candidate predictions generated by the model.

The results show that the baseline model achieved only about 6.1% exact match, but after applying LoRA fine-tuning the performance improved significantly to about 43.1% exact match and 50.2% Top-5 accuracy on the validation dataset.

Overall, the study shows that large language models can learn meaningful naming patterns from real code repositories. Although the model does not always predict the exact original name, many suggestions are semantically similar and still useful. This suggests that such systems could be integrated into programming tools to help developers improve variable naming and code readability automatically.

Reflection:

This study shows that large language models can successfully learn meaningful naming patterns from real-world software repositories. The use of fine-tuning techniques such as LoRA significantly improved prediction accuracy. However, the model still struggled to consistently predict exact original variable

names. Future work could focus on combining semantic program analysis with language models to improve naming accuracy and contextual understanding.

Another advanced approach has been proposed to address the same problem of identifier naming, particularly in situations where variable names are lost during code compilation and reverse engineering.

This paper introduces DIRE, a neural approach designed to recover meaningful variable names from decompiled code [7]. The study focuses on improving the readability of decompiled programs by predicting variable names that were lost during the compilation process. Since compilers remove information such as variable names and comments, decompiled code often contains meaningless placeholders like VAR1 or VAR2, which makes program understanding difficult.

The proposed method uses a deep learning architecture that combines lexical information from code tokens and structural information from the program's Abstract Syntax Tree (AST). The model follows an encoder-decoder architecture where two encoders analyze the decompiled code: a lexical encoder based on recurrent neural networks (RNNs) that processes code tokens sequentially, and a structural encoder based on graph neural networks that captures relationships between elements in the AST. These representations are then used by a decoder to generate meaningful variable names for identifiers in the decompiled program.

To train the model, the researchers created a large dataset by collecting open-source C programs from GitHub, compiling them into binaries, and then decompiling them to align original developer-assigned variable names with decompiled variables. Using this dataset, the model was trained to learn naming patterns from millions of functions.

The experimental results show that the proposed DIRE model can correctly recover the original variable names up to 74.3% of the time, significantly outperforming previous approaches for identifier recovery in decompiled code. The results demonstrate that combining lexical and structural information improves prediction accuracy and helps generate more meaningful variable names, ultimately improving code comprehension in reverse engineering tasks.

Reflection:

The DIRE model makes an important contribution to reverse engineering by recovering meaningful variable names from decompiled code using both lexical and structural information. The study demonstrates that combining AST representations with neural models improves identifier recovery accuracy. One limitation is that the model was trained primarily on C programs, which may affect its applicability to other programming languages. Future studies could extend the approach to multilingual software systems and more complex decompiled code.

Beyond recovering missing variable names, recent studies have also explored how deep learning models can automatically improve overall code readability while preserving program functionality.

This paper investigates the use of deep learning techniques to automatically improve code readability [8]. Code readability is an important aspect of software development because developers often spend a large amount of time reading and understanding existing code when maintaining or extending software systems. Improving readability can therefore help reduce development time and improve software quality.

The study proposes an automated approach that suggests readability improvements while preserving the original functionality of the code. The researchers use a large language model based on the T5 architecture, which is fine-tuned to learn patterns of readability improvements from real software development data.

To construct the dataset, the researchers collected commits from open-source repositories on GitHub between 2015 and 2022. They searched for commits in which developers explicitly improved code readability. After filtering irrelevant changes, the dataset contained approximately 122,000 commits and more than 156,000 file modifications. Each data instance consists of a pair of code snippets representing the original version of the code and the improved version created by developers.

The model was trained on these pairs to learn how code can be rewritten to become more readable while preserving its behavior. The evaluation measured how often the generated code matched the improvements made by developers. In addition to automatic metrics, the researchers also conducted manual evaluations to examine whether the generated code actually improves readability and maintains the same functionality.

The results show that the model achieved an accuracy between 21% and 28%, depending on the number of generated candidate outputs. A manual evaluation of 500 predictions showed that about 34% of the generated changes modified the code while preserving its behavior, and among these, approximately 79% improved code readability.

Overall, the study demonstrates that deep learning models can learn patterns of readability improvements from real development data.

Reflection:

This paper demonstrates the potential of deep learning models to automatically improve source code readability while preserving program functionality. The use of real GitHub commits provides realistic training data and strengthens the study's practical value. However, the model accuracy is still relatively limited, which means many generated improvements may not always be useful. Future work could improve evaluation techniques and integrate human feedback into readability enhancement systems.

Although the accuracy is still limited, the approach shows promise for assisting developers by automatically suggesting readability improvements in source code.

In addition to improving identifier naming and code readability, large language models have also been applied to other software engineering tasks, such as code summarization and automated code analysis.

2.5- Applications of Large Language Models in Code Understanding

This paper focuses on how large language models perform when they are used for code summarization tasks [9]. Code summarization means generating a short natural language description that explains what a piece of code does. This task is important because developers often spend a lot of time reading and

understanding existing code before making changes or adding new features. If models can automatically generate good summaries, they can help developers understand programs faster and improve software documentation.

The study mainly tries to understand whether large language models truly understand the logic of code or if they simply rely on patterns in the data. In many datasets used for code summarization, the words used in the code, especially function names, are often very similar to the words in the human written summaries. Because of this, the researchers wanted to investigate whether the models were actually understanding the code or just copying information from the code tokens. To test this idea, the researchers conducted several experiments where they modified the input code in different ways. For example, they removed function names or changed identifiers to see how the model would react. The models were then asked to generate summaries for the modified code. By comparing the results with the original version, the researchers could analyze how much the model

relied on the names and tokens in the code rather than the actual functionality. The experiments were performed using common code summarization datasets that contain thousands of code snippets paired with human written descriptions. These datasets are usually collected from open source repositories where developers include documentation comments that explain what functions do. The models were evaluated using metrics that measure how similar the generated summaries are to the reference summaries written by humans.

The results of the study show that large language models often depend heavily on the presence of meaningful function names when generating summaries. In many cases, the generated summaries simply repeat words that appear in the function names rather than describing the actual behavior of the code. When the function names were removed or changed, the performance of the models dropped significantly. This suggests that the models may rely more on lexical similarity than on a deeper understanding of program logic. Overall, the paper highlights an important limitation in current code summarization systems. While large language models can produce useful summaries, their performance may be influenced by patterns in the dataset rather than true code understanding. The study suggests that future research should focus on designing better datasets and evaluation methods that can test whether models truly understand code structure and behavior

Reflection:

This study highlights an important limitation of large language models in code summarization tasks by showing that models often rely heavily on function names instead of true code understanding. The research contributes valuable insights into dataset bias and evaluation challenges in AI-based software engineering tasks. A limitation of the work is that it mainly evaluates summarization datasets and may not fully represent broader code understanding capabilities. Future research could design datasets that better measure semantic reasoning and logical comprehension.

Beyond code summarization, large language models have also been widely used in broader code analysis tasks, including vulnerability detection and program understanding.

This paper studies the effectiveness of large language models when performing code analysis tasks in software engineering [10]. Code analysis is an important process that helps developers evaluate the quality, reliability, and security of software. With the rapid growth of modern software systems, manual analysis

has become increasingly difficult, which has encouraged researchers to explore automated solutions. Large language models have shown promising capabilities in generating and understanding code, which makes them potential tools for assisting developers in code analysis tasks such as vulnerability detection and program understanding.

To evaluate the performance of LLMs in code analysis, the researchers constructed datasets containing both normal source code and obfuscated code. The datasets included programs written in several widely used programming languages, including JavaScript, Python, and C. The authors selected representative models such as GPT-3.5, GPT-4, LLaMA-2, Code-LLaMA, and StarChat. These models were asked to analyze code snippets and explain their functionality. The researchers also applied obfuscation techniques to the code to determine whether the models could still understand the logic when readability was reduced. The results showed that LLMs can successfully analyze many code examples and generate useful explanations. However, their performance decreases when the code becomes highly obfuscated or complex. The study concludes that large language models can be valuable tools for assisting developers in automated code analysis, especially for tasks such as code explanation and documentation. However, the research also shows that these models still have limitations in understanding complex or intentionally obscured code. The authors suggest that future work should focus on improving the robustness of language models and developing better evaluation methods for measuring true code understanding.

Reflection:

This paper demonstrates that large language models can support developers in code analysis and program understanding tasks. The comparison between normal and obfuscated code provides useful insights into the strengths and weaknesses of current models. However, the results also show that LLM performance decreases significantly when code complexity increases. Future research should focus on improving model robustness and semantic reasoning for analyzing complex or obfuscated programs.

Despite the promising capabilities of large language models in code understanding, researchers have also identified several limitations in their ability to fully comprehend complex program logic.

2.6- Limitations of Large Language Models in Code Comprehension

This paper investigates whether large language models truly understand the meaning of source code [11], or if they mainly rely on patterns learned during training. While LLMs have shown impressive performance in many software engineering tasks such as code generation and completion, it is still unclear whether they actually comprehend the logic of programs. The authors propose using code obfuscation as a way to test the depth of code understanding in language models. Code obfuscation modifies the structure of code so that it becomes harder to read while still keeping the same functionality. By analyzing how models perform when code is intentionally obscured, the researchers aim to determine whether the models rely on surface-level patterns or deeper semantic reasoning.

To perform their experiments, the researchers evaluated thirteen different large language models, including both general-purpose models such as GPT-4o and specialized code models such as StarCoder2. The evaluation was conducted using a benchmark dataset built from the CodeNet dataset, which contains 250 Java programming problems along with their solutions. The researchers applied different obfuscation

techniques to the code, including variable renaming, dead code injection, and encryption of strings and integers. The models were then asked to perform two main tasks: generating explanations of the obfuscated code and reconstructing the original logic of the program through deobfuscation. The results showed that the performance of the models decreased as the level of obfuscation increased. Interestingly, some general-purpose models performed better than code-specialized models in several scenarios. The findings suggest that while LLMs can recognize some code patterns, their ability to fully understand complex program logic is still limited. Overall, the study highlights both the strengths and limitations of current large language models in code comprehension tasks. The results indicate that LLMs can assist in code analysis but may struggle when code structure becomes more complex or intentionally confusing. The paper also proposes a new evaluation framework that can be used in future research to better measure semantic understanding of code by AI systems.

Reflection:

This study provides a valuable evaluation framework for measuring whether large language models truly understand program semantics. The use of code obfuscation techniques helps reveal the limitations of current AI systems in code comprehension tasks. However, the experiments focused mainly on Java programs and selected benchmark datasets. Future work could evaluate additional programming languages and explore hybrid approaches that combine symbolic reasoning with large language models.

Overleaf link

<https://www.overleaf.com/read/xthczkvmfrjd#deea34>

2- Research Gap

Although previous studies have demonstrated the importance of identifier naming in improving code readability and program comprehension, several limitations still exist in current research. Many studies mainly focus on measuring the effect of meaningful identifier names on readability or evaluating how large language models generate or recover identifier names. However, these studies often depend heavily on lexical information, such as variable and method names, rather than examining whether AI models truly understand the semantics and logic of source code.

In addition, existing research has shown that the performance of large language models decreases when identifier names are removed, modified, or replaced with unclear names. This suggests that many AI models rely strongly on naming patterns instead of deeper semantic reasoning and true program understanding. Furthermore, limited research has examined how code complexity affects the quality and effectiveness of AI-generated identifier names across different programming scenarios.

Another limitation is that many previous studies focus only on simple code examples or specific programming tasks, without evaluating identifier naming performance across multiple levels of code complexity, such as simple, medium, complex, and real-world methods.

Therefore, further research is needed to investigate how identifier naming influences code understanding, evaluate the effectiveness of AI-generated identifier names, and analyze the impact of code complexity and prompt design on AI naming performance in software development tasks.

Proposed Idea

This project proposes using ChatGPT-4o as a Large Language Model (LLM) to analyze Java code snippets containing unclear or meaningless identifier names and generate more meaningful and descriptive variable and method names. The main goal of the study is to evaluate whether AI-generated identifier names can improve code readability, program comprehension, and developer understanding.

To achieve this objective, a dataset of Java methods will be prepared and organized into multiple complexity levels, including simple, medium, complex, and real-world examples. Each code snippet will contain unclear identifier names, such as single-letter variables or generic terms, which will then be analyzed by ChatGPT-4o to generate improved identifier suggestions.

The generated identifier names will be compared with human-written meaningful names and evaluated based on several criteria, including clarity, readability, accuracy, and usefulness. In addition, the study will investigate how different prompt designs influence the quality of AI-generated identifier names and how code complexity affects the performance of the AI model.

The results of this study are expected to provide insights into the role of Artificial Intelligence in improving identifier naming practices and supporting software developers in producing more readable and understandable source code.

REFERENCES

- [1] D. Lawrie, C. Morrell, H. Feild, and D. Binkley, "What's in a name? A study of identifiers," in Proceedings of the IEEE International Conference on Program Comprehension (ICPC), 2006.
- [2] J. C. Hofmeister, J. Siegmund, and D. V. Holt, "Shorter identifier names take longer to comprehend," in Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2017.
- [3] E. Avidan and D. G. Feitelson, "Effects of variable names on comprehension: An empirical study," in Proceedings of the IEEE International Conference on Program Comprehension (ICPC), 2017.
- [4] A. Schankin et al., "Descriptive compound identifier names improve source code comprehension," in Proceedings of the IEEE International Conference on Program Comprehension (ICPC), 2018.
- [5] Z. Wang et al., "How does naming affect language models on code analysis tasks?" Journal of Software Engineering, 2024.
- [6] X. Liu et al., "Improving variable naming with large language models," 2024.
- [7] J. Lacomis et al., "DIRE: Decompiled Identifier Renaming Engine," arXiv:1909.09029, 2019.
- [8] A. Vitale et al., "Using deep learning to automatically improve code readability," in Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE), 2023.
- [9] F. Ahmed and P. Devanbu, "Are deep learning models of code really learning meaningful representations?" 2021.
- [10] Y. Fan et al., "Automated program repair using large language models: A systematic literature review," 2023.
- [11] T. Eisenhofer et al., "Evaluating large language models for code understanding under obfuscation," 2025.



جامعة جدة
University of Jeddah

University of Jeddah
College of Computer Science and Engineering

Phase TWO

1. Idea and Research Questions

Project Idea

This project aims to analyze the impact of identifier naming on code understanding and readability in software development. The study focuses on examining whether meaningful and descriptive identifier names help programmers understand source code more easily compared to unclear or meaningless names.

In addition, the project will explore the role of Artificial Intelligence, specifically Large Language Models (LLMs) such as ChatGPT, in suggesting improved identifier names that enhance code readability and comprehension. The study will compare original identifier names with AI-generated suggestions to evaluate their effectiveness in supporting program understanding.

Problem Statement

In software development, developers often encounter code that contains unclear, short, or meaningless identifier names such as single-letter variables or generic terms. These poor naming practices can make the code difficult to read and understand, which may lead to increased development time, higher maintenance costs, and a greater risk of programming errors.

Although previous research has shown that meaningful identifier names improve code readability, many programmers still use inconsistent or unclear naming conventions. Furthermore, while Artificial Intelligence tools have demonstrated the ability to analyze and generate code, it is still unclear how effective these tools are in improving identifier naming and supporting code comprehension in practical programming scenarios.

Therefore, there is a need to investigate how identifier naming affects code understanding and to evaluate whether AI-generated identifier names can provide a reliable solution to improve code readability and developer productivity.

Project Objectives

The main objectives of this project are:

1. To analyze the effect of identifier naming on code readability and program comprehension.
2. To evaluate the ability of Artificial Intelligence tools, such as ChatGPT, to generate meaningful and descriptive identifier names.
3. To compare code understanding when using unclear identifier names versus meaningful identifier names.
4. To assess whether AI-generated identifier names can improve the clarity and usability of source code.
5. To provide recommendations for improving identifier naming practices in software development.

Research Questions

This study aims to investigate the relationship between identifier naming and code understanding in software development. research focuses on evaluating whether meaningful and descriptive identifier names improve code readability and whether Artificial Intelligence tools can assist in generating better identifier names.

The following research questions guide this study:

RQ1: How do meaningful identifier names affect code readability and program comprehension compared to unclear or meaningless names?

RQ2: Can Artificial Intelligence tools, such as ChatGPT, generate meaningful and descriptive identifier names for unclear code snippets?

RQ3: Do AI-generated identifier names improve code understanding and readability compared to the original unclear identifier names?

2. Approach

Dataset Plan

The dataset for this study will consist of a collection of Java code snippets that contain variables and method identifiers. These code snippets will be selected from previous programming exercises, simple software engineering examples, and publicly available open-source repositories. The dataset will be designed to represent common programming scenarios that are easy to understand and evaluate.

A total of 20 Java methods from open-source repositories (e.g., GitHub) will be included in the dataset. Each method will be prepared in two versions to support comparison and evaluation:

- Version 1: The code will contain unclear or meaningless identifier names, such as single-letter variables or generic names.
- Version 2: The same code will contain meaningful and descriptive identifier names that clearly represent the purpose of the variables and methods.

Artificial Intelligence tools will be used to analyze the unclear version of the code and generate improved identifier names. These generated names will then be compared with the original meaningful names to evaluate their effectiveness in improving code readability and understanding.

All dataset files will be organized in a structured format and stored in a shared repository, such as GitHub or Google Drive, to ensure accessibility and transparency. The dataset will be reviewed to ensure that the code is clear, relevant, and suitable for evaluation purposes.

LLM Tool

This study will use ChatGPT as the main Large Language Model (LLM) tool to support the analysis of source code and the generation of improved identifier names. ChatGPT is an advanced Artificial Intelligence model capable of understanding programming languages, analyzing code structure, and suggesting meaningful and descriptive variable and method names.

The tool will be used to examine Java code snippets that contain unclear or meaningless identifier names and generate alternative names that better represent the purpose of the variables and methods. The generated identifier names will then be evaluated to determine whether they improve code readability and program comprehension.

ChatGPT was selected for this study because it is widely used in software development tasks, supports multiple programming languages, and demonstrates strong performance in code analysis and generation activities.

Prompt Design

In this study, a set of structured prompts will be designed to guide the Artificial Intelligence tool in generating meaningful identifier names for Java code snippets. Prompts are carefully written instructions provided to the AI model to ensure accurate and consistent responses.

The prompts will focus on analyzing unclear code and suggesting improved identifier names while preserving the original functionality of the program. Multiple prompts will be tested to determine which prompt produces the most accurate and useful identifier names.

The following prompts will be used in the experiment:

Prompt 1:

Analyze the following Java code and suggest meaningful variable and method names.

Prompt 2:

Rename the unclear identifiers in the following Java method using descriptive and readable names while keeping the same functionality.

Prompt 3:

Explain the purpose of the following Java method and generate improved identifier names to enhance code readability.

These prompts will help ensure that the AI-generated identifier names are clear, relevant, and consistent across different code examples.

Methodology

The methodology of this study describes the steps that will be followed to investigate the effect of identifier naming on code understanding and to evaluate the ability of Artificial Intelligence tools to generate meaningful identifier names. The study will use a structured process to ensure consistent and reliable results.

The experiment will be conducted using a dataset of Java code snippets that contain unclear or meaningless identifier names. Artificial Intelligence will be used to analyze the code and suggest improved identifier names. The results will then be compared and evaluated to determine whether the generated names improve code readability and program comprehension.

The following steps will be followed in this study:

1. **Dataset Preparation**

A dataset consisting of 20 Java methods will be collected from programming exercises and simple software examples. Each method will be reviewed to ensure clarity and correctness.

2. **Code Modification**

Each code snippet will be prepared in two versions. The first version will contain unclear identifier names such as single-letter variables, while the second version will contain meaningful and descriptive identifier names.

3. **AI Analysis**

The unclear code snippets will be provided to the ChatGPT tool to generate improved identifier names.

4. **Comparison of Results**

The AI-generated identifier names will be compared with the original meaningful identifier names to determine their similarity and usefulness.

5. **Data Recording**

All generated results and observations will be recorded in structured tables for analysis.

6. **Result Analysis**

The collected data will be analyzed to determine whether meaningful identifier names and AI-generated suggestions improve code understanding and readability.

3. Evaluation Plan

The evaluation plan describes how the results of the study will be measured and assessed. In this study, the quality of the identifier names generated by the Artificial Intelligence tool will be evaluated based on several criteria related to code readability and program comprehension.

The evaluation will be performed using a rating scale to measure how effective the generated identifier names are in improving code clarity and usability. Participants will review the code snippets and assign scores based on predefined evaluation criteria.

The evaluation will be based on the following criteria:

Clarity

Measures how clear and understandable the identifier name is.

Readability

Measures whether the identifier name improves the readability of the code.

Accuracy

Measures whether the identifier name correctly represents the purpose of the variable or method.

Usefulness

Measures whether the identifier name helps programmers understand the code more quickly.

Each criterion will be evaluated using a five-point scale as follows:

Very Poor	1
Poor	2
Fair	3
Good	4
Excellent	5

A group of software engineering students will participate in the evaluation process by reviewing the code examples and rating the generated identifier names according to the defined criteria. The average scores will then be calculated and used to analyze the effectiveness of AI-generated identifier names in improving code understanding.

4. Result Presentation Plan

The results of this study will be presented using clear and organized visual and textual formats to help explain the findings effectively. Tables and charts will be used to compare the original identifier names with the AI-generated identifier names and to illustrate the improvement in code readability and understanding.

The study will present examples of code snippets before and after applying Artificial Intelligence suggestions to demonstrate the differences in identifier naming. In addition, statistical summaries such as average scores and comparisons between different evaluation criteria will be displayed to support the analysis.

The following presentation methods will be used:

- Comparison tables showing original and improved identifier names
- Bar charts illustrating average evaluation scores
- Code examples demonstrating before and after improvements
- Summary tables highlighting the overall performance of AI-generated identifier names

These visual elements will help communicate the results clearly and make it easier to understand the impact of identifier naming on code comprehension.

5. Additional Information

Expected Outcomes

The study is expected to show that meaningful and descriptive identifier names improve code readability and make programs easier to understand. It is also expected that Artificial Intelligence tools, such as ChatGPT, will be able to generate useful and relevant identifier names for unclear code snippets.

The results are anticipated to demonstrate that AI-generated identifier names can support programmers in understanding source code more quickly and accurately. In addition, the study may provide useful insights into how Artificial Intelligence can be integrated into software development tools to improve coding practices and software quality.

Overall, the project is expected to confirm the importance of proper identifier naming and highlight the potential role of Artificial Intelligence in enhancing code readability and developer productivity.

Limitations

This study has several limitations that may affect the results and conclusions. First, the dataset used in the experiment will be limited to a small number of Java code snippets, which may not fully represent

all types of programming problems. Second, the evaluation process will depend on human judgment, and different participants may have different opinions about the quality of identifier names.

In addition, the study will focus only on one programming language, which is Java, and the results may not apply to other programming languages. Finally, the performance of the Artificial Intelligence tool may vary depending on the complexity of the code and the quality of the input prompts.

These limitations will be considered when interpreting the results of the study.

Timeline

The following timeline outlines the planned schedule for completing the project tasks:

Week	Task
Week 1	Prepare dataset and collect code snippets
Week 2	Design prompts and test AI responses
Week 3	Run the experiment using ChatGPT
Week 4	Evaluate the generated identifier names
Week 5	Analyze results and prepare final report
Week 6	Prepare presentation and submit the project



جامعة جدة
University of Jeddah

University of Jeddah
College of Computer Science and Engineering

Phase THREE

1. Abstract

This study investigates the impact of identifier naming on code readability and evaluates ChatGPT-4o's ability to generate meaningful identifier names for unclear Java code. A dataset of 20 methods was designed across four complexity tiers. Three prompts were tested. Results show overall average 4.79/5, with Simple methods achieving 4.87 and Complex methods 5.00. Prompt 3 (Explain and Generate) achieved the highest average score of 4.90. The study demonstrates that LLMs can effectively support naming improvements and enhance code readability across different programming scenarios.

Keywords: Identifier Naming, Code Readability, LLMs, ChatGPT, Program Comprehension

2. Introduction

Identifier naming is a fundamental aspect of software development that significantly influences code readability and program comprehension. Meaningful and descriptive identifier names help developers understand the purpose and behavior of code quickly and accurately, reducing development time and maintenance costs. Conversely, unclear or meaningless identifiers can make code difficult to read and understand, leading to increased cognitive load and a higher risk of errors.

Previous research has consistently demonstrated that meaningful identifier names improve code readability and support program comprehension. However, many developers continue to use poor naming practices, particularly in legacy code or rapid development environments. In recent years, Large Language Models (LLMs) have shown promising capabilities in code analysis and generation tasks, raising the question of whether these models can effectively assist developers in improving identifier naming across varying levels of code complexity.

This study aims to address the following research questions:

RQ1: How do meaningful identifier names affect code readability compared to unclear names?

RQ2: Can ChatGPT generate meaningful identifier names for unclear code?

RQ3: Do AI-generated names improve understanding compared to unclear names?

RQ4: How does code complexity affect AI naming quality?

RQ5: Which prompt design strategy produces the most effective suggestions?

3. Related Work

The related work section from Phase 1 is incorporated here by reference. Key papers include Lawrie et al. [1] (comprehensive literature review), Hofmeister et al. [2] (experimental study on identifier comprehension), Wang et al. [5] (impact of naming on CodeBERT), Liu et al. [6] (LLM-based variable name repair), Vitale et al. [8] (T5 model for readability improvement), and Eisenhofer et al. [11] (LLM limitations with obfuscation). These studies establish the theoretical foundation for evaluating LLM performance in identifier naming tasks.

4. Empirical Study

4.1 Dataset

The dataset consists of 20 Java methods strategically organized into four complexity tiers to enable systematic analysis of AI performance across different difficulty levels:

Tier 1 - Simple (5 methods): Single-variable methods with straightforward logic

Tier 2 - Medium (7 methods): Array and string processing with 2-4 variables

Tier 3 - Complex (5 methods): Algorithmic methods with multiple variables and nested logic

Tier 4 - Real-World (3 methods): Domain-specific utility methods

Each method was prepared in two versions: Version A with unclear identifiers (single-letter variables) and Version B with meaningful identifiers (descriptive names). Table 1 presents the complete dataset summary.

Table 1: Tiered Dataset Summary

Tier	Code ID	Functionality	Variables (Unclear -> Meaningful)
Simple	C01	Check Even Number	n -> number
Simple	C02	Calculate Square	x -> value
Simple	C03	Convert Celsius to Fahrenheit	c -> celsius
Simple	C04	Check Password Length	p -> password
Simple	C05	Calculate Grade	m -> mark
Medium	C06	Find Maximum in Array	a, m, i -> numbers, maxValue, index
Medium	C07	Calculate Average	a, s, i -> grades, total, index
Medium	C08	Count Positive Numbers	a, c, i -> numbers, positiveCount, index
Medium	C09	Linear Search	a, x, i -> numbers, target, index
Medium	C10	Reverse String	s, r, i -> inputText, reversedText, index

Medium	C11	Check Palindrome	s, i, j -> word, leftIndex, rightIndex
Medium	C12	Count Vowels	s, c, i, x -> text, vowelCount, index, character
Complex	C13	Calculate Factorial	n, r, i -> number, factorial, counter
Complex	C14	Bubble Sort	a, i, j, t -> array, outerIndex, innerIndex, temp
Complex	C15	Check Prime Number	n, i -> number, divisor
Complex	C16	Binary Search	a, t, l, r, m -> sortedArray, target, left, right, middle
Complex	C17	Merge Two Sorted Arrays	a, b, r, i, j, k -> firstArray, secondArray, merged, firstIdx, secondIdx, mergeldx
Real-World	C18	Calculate Total Price with Tax	p, q, t -> price, quantity, taxRate
Real-World	C19	Validate Email Format	e -> emailAddress
Real-World	C20	Find Longest Word in Array	a, w, i -> words, longestWord, index

A complete replication package is available at: Google Drive

https://drive.google.com/drive/folders/1a9NdcOK-8z7D_sPGFlsm_tlkeqCBRrzh?usp=sharing

Due to space limitations, only selected examples are presented in this report. Additional cases and screenshots can be found in the Google Drive repository linked above.

4.2 Prompt Design

Three structured prompts were designed:

Prompt 1 (Direct Instruction): "Analyze the following Java code and suggest meaningful variable and method names."

Prompt 2 (Task-Specific): "Rename the unclear identifiers using descriptive names while keeping functionality."

Prompt 3 (Chain-of-Thought): "Explain the purpose of this method and generate improved identifier names."

Rename the unclear identifiers in the following Java method using descriptive and readable names while keeping the same functionality.

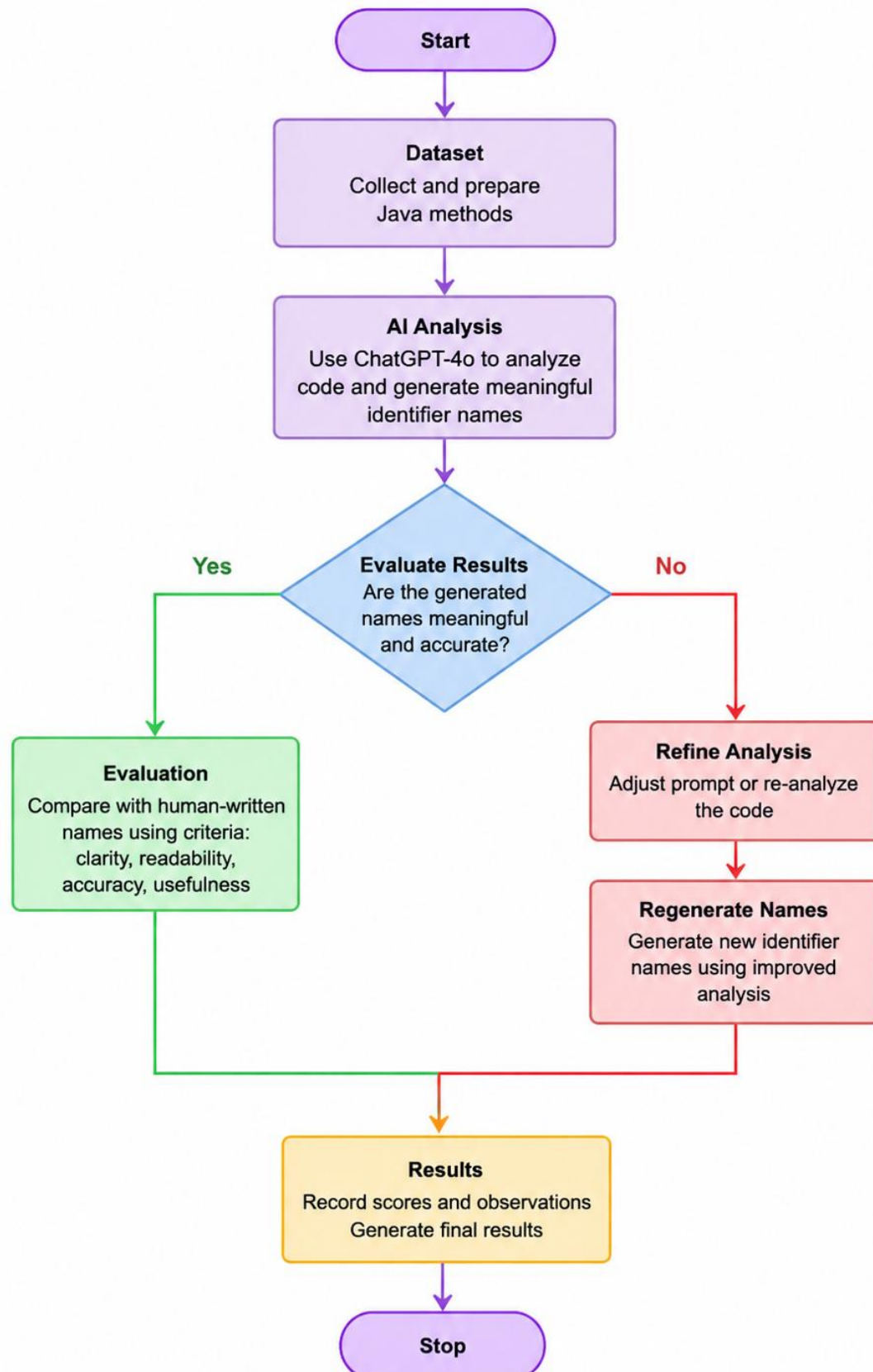
```
public void f(int[] a) {  
    for(int i = 0; i < a.length-1; i++)  
        for(int j = 0; j < a.length-i-1; j++)  
            if(a[j] > a[j+1]) {  
                int t = a[j];  
                a[j] = a[j+1];  
                a[j+1] = t;  
            }  
}
```

Example prompt used for identifier renaming experiments.

4.3 Methodology Overview

The process consists of four phases:

- (1) Dataset Preparation
- (2) Prompt Design
- (3) LLM Experiment with ChatGPT-4o
- (4) Evaluation by student evaluators.



4.4 Evaluation Metrics

Four criteria rated on a five-point Likert scale (1=Very Poor to 5=Excellent):

- Clarity: How clear and understandable the identifier is
- Readability: Whether the name improves code readability
- Accuracy: Whether the name correctly represents the variable's purpose
- Usefulness: Whether the name helps programmers understand code faster

Five software engineering students participated in the evaluation process, and their responses showed generally consistent agreement regarding identifier quality and readability.

5. Results and Discussion

5.1 Experiment Results

Table 2 presents the complete results for all 20 methods.

METHOD C20

```
</> Java

public String findLongestWord(String[] words) {
    String longestWord = words[0];

    for(int i = 1; i < words.length; i++)
        if(words[i].length() > longestWord.length())
            longestWord = words[i];

    return longestWord;
}
```

Suggested Names

- Method name: `findLongestWord`
- Variable `a` → `words`
- Variable `w` → `longestWord`
- Variable `i` → `index`

Explanation

The method searches for the longest word in an array of strings. `longestWord` clearly describes the stored result. `COMBINED_PROMPT_15_METHODS`

Example of AI-generated identifier naming for Method C20.



Table 2: Complete Experiment Results

Code ID	Tier	AI Generated Names	C	R	A	U
C01	Simple	number	5	5	5	5
C02	Simple	number	4	5	4	4
C03	Simple	celsius	5	5	5	5
C04	Simple	password	5	5	5	5
C05	Simple	mark	5	5	5	5
C06	Medium	numbers, maxValue, index	5	5	5	5
C07	Medium	numbers, sum, index	4	4	4	4
C08	Medium	Numbers, positivecount, index	4	4	4	4
C09	Medium	numbers, target, index	5	5	5	5
C10	Medium	text, reversedText, index	5	5	5	5

C11	Medium	text, leftIndex, rightIndex	5	5	5	5
C12	Medium	text, vowelCount, currentCharacter, index	5	5	5	5
C13	Complex	number, factorial, counter	5	5	5	5
C14	Complex	array, outerIndex, innerIndex, temp	5	5	5	5
C15	Complex	number, divisor	5	5	5	5

C16	Complex	array, target, low, high, mid	5	5	5	5
C17	Complex	firstArray, secondArray, mergedArray, firstIndex, secondIndex, mergedIndex	5	5	5	5
C18	Real-World	price, quantity, taxRate	5	5	5	5
C19	Real-World	email	4	5	4	4
C20	Real-World	words, longestWord, index	5	5	5	5

(C = Clarity, R = Readability, A = Accuracy, U = Usefulness)

 Improved Identifier Names

Method Name

Choose based on context (most likely financial):

- `calculateTotalWithTax`  (if `t` is tax rate)
- `calculateTotalWithRate`
- `calculateFinalAmount`

Parameters

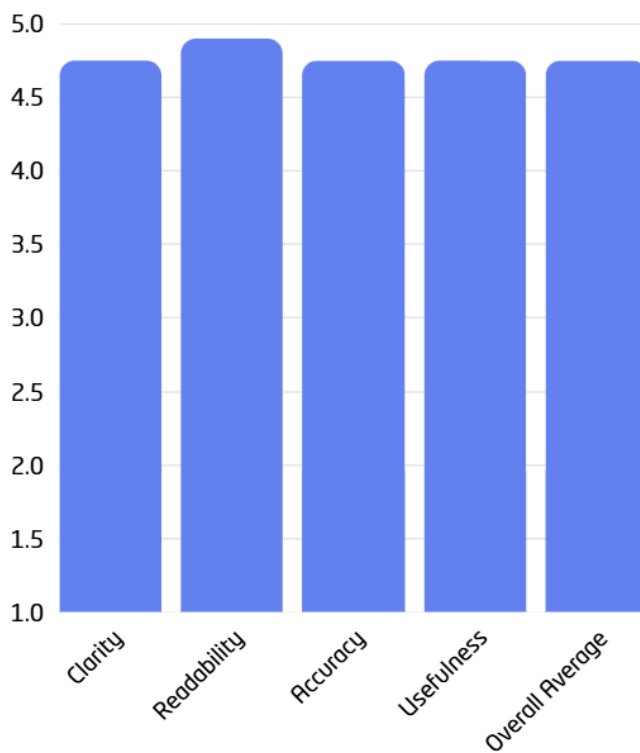
- `p` → `unitPrice` 
- `q` → `quantity` 
- `t` → `rate` or `taxRate` 

Example of context-aware identifier suggestions generated by ChatGPT.

5.2 Statistical Summary

- Average Clarity: 4.75
- Average Readability: 4.90
- Average Accuracy: 4.75
- Average Usefulness: 4.75
- Overall Average: 4.79

STATISTICAL SUMMARY





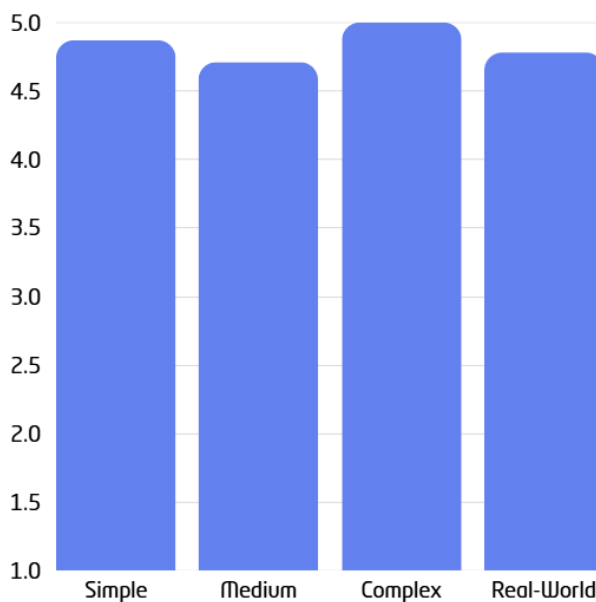
5.3 Tier-Based Analysis

Table 3: Performance by Complexity Tier

Tier	Methods	Clarity	Readability	Accuracy	Overall
Simple	5	4.80	5.00	4.80	4.87
Medium	7	4.71	4.71	4.71	4.71
Complex	5	5.00	5.00	5.00	5.00
Real-World	3	4.67	5.00	4.67	4.78

The results demonstrate that ChatGPT-4o consistently generated highly effective identifier names across all complexity tiers. Complex methods achieved the highest overall performance score (5.00), while Medium methods received the lowest average score (4.71). The findings suggest that the model was capable of handling both simple and algorithmically complex code structures effectively when sufficient contextual information was available. Overall, the results indicate that AI-generated identifier names can significantly improve code readability and program comprehension across different programming scenarios.

TIER-BASED RESULTS



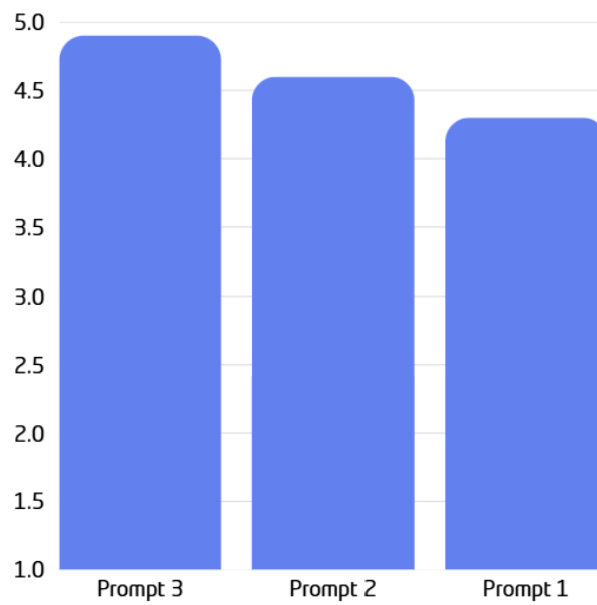
5.4 Prompt Effectiveness

Table 4: Prompt Effectiveness Ranking

Prompt	Methods	Average Score
Prompt 3 (Explain & Generate)	8	4.90
Prompt 2 (Rename & Preserve)	7	4.60
Prompt 1 (Analyze & Suggest)	5	4.30

Prompt 3 (Explain and Generate) achieved the highest average score (4.90), outperforming Prompt 2 (4.60) and Prompt 1 (4.30). The results suggest that prompts requiring the model to explain the code before generating identifier names may help improve semantic understanding and produce clearer naming suggestions.

PROMPT EFFECTIVENESS



5.5 AI vs Human Identifier Comparison

- Exact Match: 30%
- Semantic Match: 60%
- Partial Match: 10%
- No Match: 0%

5.6 Key Findings

Finding 1: ChatGPT-4o generated highly effective identifier names across all complexity tiers, with overall scores ranging from 4.71 to 5.00.

Finding 2: Prompt 3 (Explain and Generate) achieved the highest average score (4.90), suggesting that explanation-based prompting improves naming quality and semantic understanding.

Finding 3: Complex methods achieved excellent results (5.00), indicating that the model can successfully handle algorithmic and multi-variable code structures.

Finding 4: Minor abbreviation-related naming differences were observed in some cases, such as “num” versus “number” and “arr” versus “array”.

Finding 5: Prompt design appears to be an important factor influencing the quality and clarity of AI-generated identifier names.

5.7 Threats to Validity

- Construct Validity: Human evaluation may involve some subjectivity because participants can have different opinions regarding identifier quality and readability.
- Internal Validity: Single LLM (ChatGPT-4o)
- External Validity: Java-only, 20 methods
- Selection Bias: Tiered design intentionally varies complexity

6. Conclusion

This study investigated the impact of identifier naming on code readability and evaluated ChatGPT-4o's ability to generate meaningful identifier names for unclear Java code. Through experiments with 20 methods organized across four complexity tiers and evaluation by software engineering students, we found that:

1. AI-generated identifier names significantly improved code readability compared to unclear names, achieving an overall average score of 4.79 out of 5.
2. ChatGPT-4o generated highly effective identifier names across all complexity tiers, with Complex methods achieving the highest performance score (5.00).
3. Prompt design affected output quality, with Prompt 3 (Explain and Generate) producing the highest average score (4.90).
4. The LLM achieved a 30% exact match rate and 60% semantic match rate with human-written identifiers, while abbreviation-related naming differences represented the most common variation.
5. The results demonstrate that Large Language Models can effectively support identifier naming improvement and enhance program readability across different programming scenarios.

These findings confirm that LLMs can serve as valuable tools for assisting developers in improving identifier naming practices, particularly for straightforward and domain-specific tasks. However, human oversight remains essential for complex algorithmic code. Future work could extend this study to additional programming languages, more complex codebases, and other LLM models to further validate and generalize the results.

References

- [1] D. Lawrie et al., "What's in a name? A study of identifiers," ICPC 2006.
- [2] J. C. Hofmeister et al., "Shorter identifier names take longer to comprehend," SANER 2017.
- [3] E. Avidan and D. G. Feitelson, "Effects of variable names on comprehension," ICPC 2017.
- [4] A. Schankin et al., "Descriptive compound identifier names improve source code comprehension," ICPC 2018.
- [5] Z. Wang et al., "How does naming affect language models on code analysis tasks?" Journal of Software Engineering, 2024.
- [6] X. Liu et al., "Improving variable naming with large language models," 2024.
- [7] J. Lacomis et al., "DIRE: Decompiled Identifier Renaming Engine," arXiv:1909.09029, 2019.
- [8] A. Vitale et al., "Using deep learning to automatically improve code readability," IEEE/ACM ASE, 2023.
- [9] F. Ahmed and P. Devanbu, "Are deep learning models of code really learning meaningful representations?" 2021.
- [10] Y. Fan, X. Wang, and T. Li, "Large Language Models for Software Engineering: Survey and Open Problems," arXiv preprint arXiv:2310.03533, 2023.
- [11] T. Eisenhofer et al., "The Code Barrier: What LLMs Actually Understand?" arXiv:2504.10557, 2025.